An Exact Lexicographic Min-Cost Flow Solver for the Laboratory Assignment Problem

Meteo

2026-06-13

Abstract

This document describes the mathematical model, algorithm, implementation, and verification strategy of a C program for the laboratory assignment problem. The program assigns every student to exactly one laboratory, never exceeds laboratory capacities, and assigns at least one student to every laboratory with positive capacity. The core algorithm is an exact network-flow solver: the default mode minimizes rank sum, rank squared sum, worst assigned rank, and fill-rate objectives lexicographically without using a Big-M scalarization. The implementation additionally compresses equivalent capacity slots and uses an indexed heap so that the exact algorithm remains fast on maximum-size instances. To reduce manual checking work for users, it automatically writes only the simple assignment file by default, and can optionally write numerical metrics and review reports when the `--reports` option is specified. For reproducibility and performance inspection, the implementation also provides preset rank-cost and weighted objective files, objective-printing helpers, exact counterfactual explanations, an optional solver profile TSV, and process-parallel portfolio execution.

1 Problem Statement

Let

$$S = \{1, \dots, N\}, \qquad L = \{1, \dots, M\}$$

be the set of students and laboratories. The target benchmark constraints are

$$1 \le N \le 1024, \qquad 1 \le M \le 256.$$

The implementation does not hard-code these two benchmark limits as rejection conditions. Larger feasible inputs are accepted subject to memory, time, and standard-C integer-index limits. Laboratory j has capacity $c_j \ge 0$, and the total capacity satisfies

$$\sum_{j=1}^{M} c_j \ge N.$$

Student i lists R preferred laboratories. If laboratory j is the k-th preference of student i, define $r_{ij} = k$. If j is not listed by student i, this implementation uses the outside-preference rank:

$$r_{ij} = M + 1.$$

The decision variable is

$$x_{ij} \in \{0, 1\},$$

where $x_{ij} = 1$ means that student i is assigned to laboratory j. The hard constraints are

$$\sum_{j=1}^{M} x_{ij} = 1 \quad (\forall i \in S),$$

$$\ell_j \le \sum_{i=1}^{N} x_{ij} \le c_j \quad (\forall j \in L),$$

where

$$\ell_j = \begin{cases} 1 & (c_j > 0), \\ 0 & (c_j = 0). \end{cases}$$

If the number of positive-capacity laboratories is greater than N, these constraints are infeasible and the program reports an input error.

2 Evaluation Metrics

For a feasible assignment x, define

$$Q(x) = \max_{i,j:x_{ij}=1} r_{ij},$$

$$R_1(x) = \sum_{i=1}^{N}\sum_{j=1}^{M} r_{ij}x_{ij}, \qquad R_2(x) = \sum_{i=1}^{N}\sum_{j=1}^{M} r_{ij}^2 x_{ij}.$$

Minimizing $R_1(x)$ is equivalent to minimizing the average assigned rank. For assignments with fixed $R_1(x)$, minimizing $R_2(x)$ is equivalent to minimizing the rank variance because

$$\sigma^2(x) = \frac{R_2(x)}{N} - \left(\frac{R_1(x)}{N}\right)^2.$$

For positive-capacity laboratories, let $n_j = \sum_i x_{ij}$. The minimum fill rate is

$$U_{\min}(x) = \min_{j:c_j>0} \frac{n_j}{c_j}.$$

The average fill rate is

$$U_{\text{avg}}(x) = \frac{1}{|\{j : c_j > 0\}|} \sum_{j:c_j>0} \frac{n_j}{c_j}.$$

3 Optimization Order

The evaluation setup does not define one absolute scalar score. It gives five separate metrics whose relative rankings are averaged. Therefore the default solver mode uses a rubric-aligned deterministic Pareto-lexicographic objective:

$$(R_1,\ R_2,\ Q,\ -U_{\text{avg}},\ -U_{\min}).$$

All five components are optimized exactly under this order. This does not claim to predict an external final average ranking against other solvers, because other solvers' outputs are unknown. It does guarantee that the program does not return a solution that is worse under the stated lexicographic objective.

The implementation also keeps the earlier conservative fairness order as `--objective fair`:

$$Q,\ R_1,\ R_2,\ -U_{\min},\ -U_{\text{avg}}.$$

The `balanced` mode uses

$$R_1,\ R_2,\ Q,\ -U_{\min},\ -U_{\text{avg}},$$

and `guarded` first bounds Q by the minimum feasible value plus a user-specified slack, then optimizes the default rubric order inside that guard.

For practical student satisfaction, the implementation also provides a `satisfaction` objective mode. This mode replaces the raw rank r by a non-linear dissatisfaction cost $d(r)$, then optimizes

$$D_1,\ D_2,\ Q,\ -U_{\text{avg}},\ -U_{\min},$$

where

$$D_1 = \sum_i d(r_i), \qquad D_2 = \sum_i d(r_i)^2.$$

The default model is

$$d(1) = 0, \quad d(r) = 100 + 30(r-2) + 5(r-2)^2\ (2 \le r \le M), \quad d(M+1) = 10000.$$

These constants can be changed from the command line or by a rank-cost table. Because D_1 and D_2 are additive over student–laboratory edges, this mode remains a single lexicographic min-cost-flow optimization followed by the same exact Q, U_{avg}, and $U_{\min}$ tie-breaks. It does not require the two-dimensional threshold enumeration used by weighted exact optimization.

The rank-cost table can be supplied from a plain text preset file. The repository includes presets such as `student_friendly`, `balanced_satisfaction`, and `strict_outside_avoidance`. Weighted exact mode similarly accepts a plain text weight file. These files do not change the solver; they only make the exact objective configuration auditable and easy to reproduce.

The `fill-convex` mode adds another exact candidate for operational balance. Let $g_j = \min(c_j, N)$ be the graph capacity and $G = \sum_j g_j$. The capacity-proportional ideal count is

$$t_j = N\frac{g_j}{G}.$$

The mode minimizes a scalar objective combining the ordinary rank cost with the separable convex penalty

$$\sum_j (n_j G - N g_j)^2.$$

It is therefore not a rank-first tie-breaker; changing the convex penalty can change the selected assignment even when the pure rank tuple differs. This is still a min-cost-flow problem: for the x-th student assigned to laboratory j, the marginal penalty is

$$G^2(2x-1) - 2GNg_j.$$

The program expands the laboratory-to-sink capacity into one-person edges with these marginal costs. Since the marginal-cost representation is exactly the difference of consecutive squared penalties, the selected assignment is exact for this convex-balance objective.

4 Intuitive Explanation of Optimality

The main reason this method is optimal is that the program does not guess assignments one by one. Instead, it changes the assignment problem into a pipe network.

Think of every student as one unit of water. The water must leave the source, pass through exactly one student node, pass into exactly one laboratory node, and finally reach the sink. A laboratory pipe has limited capacity, so too many students cannot enter that laboratory. A positive-capacity laboratory also has required slots, so the solver is forced to use at least one slot there whenever that is mathematically possible.

This means the network has the same choices as the original assignment problem:

- every valid assignment is one complete integer flow;
- every complete integer flow is one valid assignment;

- no other hidden choices exist.

Therefore, optimizing the flow is the same as optimizing the assignment.

The default mode attaches a small report card to each possible pipe. The report card is not a single fragile number; it is an ordered tuple:

$$(\text{rank sum},\ \text{rank-square sum},\ \text{worst rank},\ \text{average fill rate},\ \text{minimum fill rate}).$$

The solver compares these tuples exactly from left to right. Thus it first minimizes the average rank, then minimizes rank unfairness, then minimizes the worst assigned rank, and only then uses exact fill-rate tie-breaking. This is the same as saying:

> First make the total rank as small as possible. If there is still a tie, make the spread of ranks as small as possible. If there is still a tie, improve the worst assigned rank, then the average laboratory fill rate, and finally the worst laboratory fill rate.

Because min-cost flow globally minimizes this tuple over all complete flows, the chosen assignment is not merely locally good. It is the best assignment under the stated lexicographic objective.

There is one important fast path in the fair-mode route. After the worst-rank threshold has been fixed, if every active student has exactly the same rank table, then exchanging two students never changes the fair-mode objective components. The only information that matters is the count n_j assigned to each laboratory. In that special case, the program skips the student-laboratory flow and optimizes the count vector directly: it fills lower-rank laboratories before higher-rank ones, then maximizes the exact minimum fill rate, and only then maximizes the exact average fill rate. Other rank-first modes still remain exact, but they use the general grouped min-cost-flow path rather than this count-only shortcut.

5 Formal Correctness Lemmas

The exactness argument used by the implementation can be audited through the following lemmas.

Lemma 1 (Assignment-flow equivalence)**.** *Every feasible assignment corresponds to one integral feasible flow in the student-laboratory network, and every integral feasible flow corresponds to one feasible assignment.*

Proof. The source sends one unit through each assigned student, each student node has capacity one from the source, and each unit then enters exactly one laboratory node before reaching the sink. Laboratory-to-sink bounds are exactly the minimum-occupancy and capacity constraints. Thus the network choices and the assignment choices are the same. □

Lemma 2 (Integrality)**.** *The flow solution can be taken to be integral.*

Proof. All capacities and lower bounds are integral. Standard max-flow and min-cost-flow algorithms on such networks return integral augmentations, so the resulting flow values are integral. □

Lemma 3 (Threshold monotonicity)**.** *If rank threshold T is feasible, every threshold $T' > T$ is feasible.*

Proof. Increasing the threshold only adds student-to-laboratory edges. It never removes a feasible flow. □

Lemma 4 (Binary search returns minimum Q)**.** *Binary search over threshold feasibility returns the minimum possible worst assigned rank Q.*

Proof. By threshold monotonicity, feasible thresholds form a suffix of $\{1, \ldots, M+1\}$. Binary search returns the first element of this suffix. □

Lemma 5 (Tuple costs optimize R_1 and R_2)**.** *With Q fixed, lexicographic tuple min-cost flow minimizes R_1, and among those assignments minimizes R_2, without Big-M scalarization.*

Proof. The second tuple component is exactly R_1, and the third tuple component is exactly R_2. Lexicographic comparison minimizes them in that order. □

Lemma 6 (Rank variance)**.** *When R_1 is fixed, minimizing R_2 minimizes rank variance.*

Proof. The variance is

$$\sigma^2 = \frac{R_2}{N} - \left(\frac{R_1}{N}\right)^2.$$

For fixed R_1 and N, only R_2 can change. □

Lemma 7 ($U_{\min}$ candidates and monotonicity)**.** *Every achievable minimum fill rate is some a/c_j, and feasibility for a candidate q is monotone downward.*

Proof. For each positive-capacity laboratory, n_j is integral, so $n_j/c_j = a/c_j$. If all laboratories satisfy $n_j/c_j \geq q$, they also satisfy every smaller candidate. □

Lemma 8 (Exact U_{avg} comparison)**.** *Least-common-denominator integer comparison orders average fill rates exactly.*

Proof. Let $L = \operatorname{lcm}\{c_j \mid c_j > 0\}$. This denominator is fixed for the input, so comparing $\sum_j n_j L/c_j$ is equivalent to comparing $\sum_j n_j/c_j$. The implementation evaluates this integer comparison with big integers, so no rounding is introduced. □

Lemma 9 (Equal-capacity bucket compression)**.** *Combining laboratories with the same positive capacity preserves U_{avg} exactly.*

Proof. For $C = \{d \mid \exists j,\ c_j = d,\ d > 0\}$,

$$\sum_{j:c_j>0} \frac{n_j}{c_j} = \sum_{d \in C} \frac{\sum_{j:c_j=d} n_j}{d}.$$

This only combines equal-denominator terms. □

Lemma 10 (Active rank-row grouping)**.** *After the optimal worst rank Q is fixed, grouping students with identical active rank rows preserves all five objective values.*

Proof. For a fixed Q, inactive edges with $r_{ij} > Q$ cannot be used by any rank-optimal assignment. If two students have the same available laboratories and the same rank value on every active edge, swapping their assigned active laboratories does not change Q, R_1, R_2, laboratory counts, $U_{\min}$, or U_{avg}. Therefore only group-to-lab counts matter inside such a group. □

Lemma 11 (Allowed-set feasibility grouping)**.** *For a fixed threshold T, grouping students with the same allowed laboratory set preserves feasibility.*

Proof. The threshold feasibility test only asks whether each student can use one laboratory from $\{j \mid c_j > 0,\ r_{ij} \leq T\}$. Rank values inside this set are irrelevant until the later min-cost-flow phases. Therefore students with the same allowed set are interchangeable for the feasibility network. A group node with capacity equal to the group size can send exactly the same number of assignment units to the same set of laboratories as the individual student nodes could. □

Lemma 12 (Capacity slot compression)**.** *Replacing identical unit slots by one capacity edge preserves feasible flows and costs.*

Proof. Sending k units through one capacity-c edge with linear cost is equivalent to sending those k units through k identical unit edges. □

6 Flow Model

The assignment is represented as a bipartite flow network.

- Source s connects to each student i with capacity 1.
- Student i connects to laboratory j with capacity 1 if the current rank threshold permits r_{ij}.
- Laboratory j connects to sink t through capacity slots.

Because all capacities are integers, integral min-cost flow returns integral flows. Thus every unit of flow corresponds exactly to assigning one student to one laboratory.

7 Rank-First Rubric Objective

The default `rubric` mode follows the evaluation-metric order:

$$R_1 \to R_2 \to Q \to U_{\text{avg}} \to U_{\min}.$$

It first solves a rank-cost min-cost-flow problem over all rank thresholds up to $M+1$, obtaining the minimum pair (R_1^*, R_2^*). Then it searches the compressed rank-threshold candidate list for the smallest Q that can still achieve (R_1^*, R_2^*). Finally it fixes the rank pair and Q, maximizes the exact average fill rate, and among those solutions maximizes the exact minimum fill rate.

Theorem 1. *The default `rubric` solution is lexicographically optimal for* $(R_1, R_2, Q, -U_{\text{avg}}, -U_{\min})$.

Proof. The first min-cost-flow run uses student–laboratory edge cost $(0, r_{ij}, r_{ij}^2)$, so its second and third total-cost components are exactly R_1 and R_2. Integral min-cost flow therefore returns the minimum achievable rank pair. Restricting to thresholds Q and re-solving with the rank pair fixed tests exactly whether that same pair is achievable under the threshold. Rank-threshold feasibility is monotone, and the program searches only thresholds where the edge set can change, so the returned threshold is the smallest Q preserving the rank pair. The fill-rate tie-breakers are then optimized with the rank pair and threshold fixed, using the exact rational comparison described below. Each phase optimizes the next component over the feasible set left by all previous phases. □

8 Worst Rank Minimization for Fair Mode

For a threshold T, keep only edges satisfying

$$r_{ij} \le T.$$

Feasibility is monotone in T. The `fair` mode uses a lower-bound max-flow feasibility test and binary search over the compressed rank-threshold candidate list. This same test also provides lower bounds and feasibility checks for other exact objectives. A laboratory edge has lower bound ℓ_j and upper bound c_j. The network groups students by the allowed laboratory set for the tested threshold, then uses the standard lower-bound transformation: an edge $u \to v$ with lower bound a and upper bound b is replaced by capacity $b-a$, and node balances are updated by

$$balance[u] \mathrel{-}= a, \qquad balance[v] \mathrel{+}= a.$$

Then a super source and super sink are added, and feasibility is equivalent to saturating all positive balance demands.

Theorem 2. *The threshold returned by the search is the minimum possible value of* $Q(x)$.

Proof. For a fixed threshold T, the transformed flow network is feasible if and only if there exists an assignment satisfying all hard constraints using only edges with $r_{ij} \le T$. If a threshold is feasible, every larger threshold is also feasible because it only adds edges. Therefore the feasible thresholds form an interval, and binary search returns the smallest feasible threshold. □

9 Lexicographic Min-Cost Flow

After the optimal threshold T^* is known, only edges with $r_{ij} \le T^*$ are used. The program stores the ordinary min-cost-flow part as a three-component tuple

$$C = (c_0, c_1, c_2),$$

ordered lexicographically. Arithmetic is componentwise and checked for signed integer overflow. This avoids Big-M weights and avoids priority collapse from scalarization.

The cost of assigning student i to laboratory j is

$$(0,\ r_{ij},\ r_{ij}^2).$$

Laboratory slots encode required minimum counts. If a slot is required, it has first component -1; otherwise its first component is 0. Therefore minimizing the first component maximizes the number of required slots used. When all required slots can be used, the remaining components determine the rank-optimal solution. The final average fill-rate component is handled separately by the exact least-common-denominator comparison in Section 11. This keeps the hot min-cost-flow path smaller while preserving the full five-component objective.

Theorem 3. *With T^* fixed, the lexicographic min-cost flow minimizes R_1, and among all assignments with minimum R_1, minimizes R_2.*

Proof. All feasible solutions after the required-slot component is satisfied have the same first component. The second component of total cost is exactly $\sum r_{ij}x_{ij} = R_1(x)$. Lexicographic minimization therefore minimizes R_1. Among solutions with the same second component, the third component is exactly $\sum r_{ij}^2 x_{ij} = R_2(x)$, so it is minimized next. □

10 Exact Fill-Rate Tie-Breaking

Different objective modes use the same exact fill-rate machinery in different orders. In `rubric`, `satisfaction`, and `guarded`, the solver first maximizes U_{avg} after the rank target and Q have been fixed, then finds the largest $U_{\min}$ threshold that preserves that same average-fill value. In `fair` and `balanced`, the minimum-fill component is the next tie-breaker before the final average-fill tie-breaker.

The minimum-fill subproblem is exact. Every possible minimum fill-rate value has the form

$$q = \frac{a}{c_j}$$

for some positive-capacity laboratory j and integer

$$1 \le a \le \min(c_j, N).$$

The program enumerates these rational candidates, sorts them by exact cross-multiplication, removes duplicates, and binary-searches the best candidate.

For a candidate q, laboratory j must receive at least

$$\lceil qc_j \rceil$$

students. The program marks exactly that many slots as required. It then runs the same lexicographic min-cost flow. The candidate is accepted if all required slots are used and the rank components R_1 and R_2 remain equal to the previously proven optima.

Theorem 4. *When the selected objective asks for minimum fill before average fill, the selected candidate maximizes $U_{\min}$ without worsening the already fixed rank and threshold components.*

Proof. The candidate set contains every value that can be achieved by n_j/c_j. For a fixed q, the slot requirements are exactly equivalent to $n_j/c_j \geq q$ for all positive-capacity laboratories. Feasibility is monotone in q: if q is feasible, every smaller candidate is feasible. Binary search over the sorted candidate set therefore returns the greatest feasible q. The acceptance test additionally requires the already fixed rank components to remain unchanged, so the earlier optima are preserved. □

11 Exact Average Fill-Rate Tie-Breaking

Whenever a mode needs to maximize U_{avg} under previously fixed components, the solver does so without using a floating-point or scaled integer approximation. Let

$$P = \{j \mid c_j > 0\}$$

and define the conceptual per-laboratory common denominator

$$L = \operatorname{lcm}\{c_j \mid j \in P\}.$$

For each positive-capacity laboratory j, define

$$W_j = \frac{L}{c_j}.$$

Then

$$U_{\text{avg}}(x) = \frac{1}{|P|} \sum_{j \in P} \frac{n_j}{c_j} = \frac{1}{|P|L} \sum_{j \in P} n_j W_j.$$

Because $|P|$ and L are fixed for the input instance, maximizing U_{avg} is exactly equivalent to maximizing the integer value

$$\sum_{j \in P} n_j W_j.$$

For implementation, laboratories with the same positive capacity are grouped. Let

$$C = \{d \mid \exists j,\ c_j = d,\ d > 0\}$$

be the set of distinct positive capacities, and let

$$B_d = \{j \mid c_j = d\}.$$

Then

$$\sum_{j : c_j > 0} \frac{n_j}{c_j} = \sum_{d \in C} \frac{\sum_{j \in B_d} n_j}{d}.$$

Thus equal-denominator terms are combined before the least-common-denominator integer comparison. This transformation is algebraically exact and only reduces the coefficient dimension.

The weights W_j can be much larger than 64-bit integers, especially when there are many laboratories. The program therefore stores them with a small fixed-limb unsigned big-integer type. After equal-capacity grouping, one weight is stored per distinct positive capacity rather than per laboratory. The final min-cost flow phase does not store a huge integer on every graph edge. Instead, each laboratory-to-sink edge carries only the capacity-term index and a coefficient -1. During shortest-path comparisons, the solver compares the accumulated coefficient vectors by evaluating the corresponding big-integer weighted sums exactly.

There is also a scalar fast path for ordinary rank-first objectives. Let

$$U_s(x) = \sum_{j \in P} n_j W_j$$

be the exact scaled fill reward and let $T(x)$ be the already lower-priority rank-square or dissatisfaction-square component. If L, all W_j, and the required arithmetic bounds fit in signed integer costs, the solver chooses

$$M > \max_{x,y} |U_s(x) - U_s(y)|$$

and replaces the final pair $(T, -U_s)$ by the single integer

$$TM - U_s.$$

If $T(x) < T(y)$, then $T(y) - T(x) \geq 1$, so the M gap cannot be reversed by any possible fill-reward difference. If $T(x) = T(y)$, minimizing $TM - U_s$ is exactly the same as maximizing U_s. Therefore this fast path preserves the same lexicographic order. If any safety check fails, the solver uses the big-integer comparison path described above.

The implementation uses two exact-preserving refinements to make this safety test less conservative. First, under a fixed max-rank threshold q, edges with rank greater than q are not inserted into the flow graph. Overflow and big-M bounds therefore need only consider rank costs for active ranks $1, \ldots, q$. Second, the fill-reward difference bound is computed from active capacity-limited reward slots. Each positive-capacity laboratory j contributes at most

$$\min(g_j, a_j)$$

slots of reward W_j, where g_j is its graph capacity and a_j is the number of students with an active edge to j under the same threshold. No assignment can send more than g_j students to j, and no assignment can send more than those a_j reachable students. Every complete assignment therefore selects exactly N slots from this capped multiset, so the sum of the largest N slot rewards is an upper bound on U_s, and the sum of the smallest N slot rewards is a lower bound. Their difference is a valid D for the proof above and is no larger than the coarser $N(\max_j W_j - \min_j W_j)$ bound.

Theorem 5 (Exact fill tie-breakers)**.** *After the rank-side prefix of a selected objective mode has been fixed, the solver applies the selected fill tie-breaker exactly. For average-first modes such as* `rubric`*,* `satisfaction`*, and* `guarded`*, it first maximizes* U_{avg}*, then maximizes* $U_{\min}$ *subject to preserving that optimal* U_{avg}*. For minimum-first modes such as* `fair` *and* `balanced`*, it first maximizes* $U_{\min}$*, then maximizes* U_{avg} *subject to preserving that optimal* $U_{\min}$*.*

Proof. At this point the earlier rank-side constraints are represented as hard conditions: the selected rank threshold and the already optimal additive rank costs must be preserved. In an average-first phase, each laboratory-to-sink unit of flow for laboratory j contributes the exact reward represented by W_j. Thus the fill component of a full flow is

$$-\sum_{j \in P} n_j W_j.$$

Minimizing this value is exactly the same as maximizing $\sum_j n_j W_j$, and by the equality above, exactly the same as maximizing U_{avg}. The subsequent minimum-fill phase adds the minimum count vector corresponding to a candidate threshold and searches for the largest threshold that can preserve the already optimal average-fill value.

In a minimum-first phase the order is reversed. The solver first searches minimum-count threshold vectors to find the largest achievable $U_{\min}$. With that vector fixed, the same exact average-fill comparison maximizes U_{avg}. In both cases, weighted sums are compared with integer arithmetic, so no rounding can change the ordering. □

11.1 Why Exact Average Comparison Is Needed Only in Fill Phases

The average fill-rate comparison is not needed while the solver is still deciding an earlier lexicographic component. If two partial costs differ in rank sum, rank-square sum, worst rank, or an already selected minimum-fill threshold, U_{avg} cannot change that lexicographic decision. Therefore the expensive big-integer comparison is used only in phases where the selected objective mode actually compares average fill among assignments whose earlier components are already tied.

For average-first modes, the exact comparison chooses the member of the rank-side optimum set with maximum U_{avg}, and a later phase chooses maximum $U_{\min}$ without losing that value. For minimum-first modes, the minimum-fill value is fixed first, and the exact comparison then chooses maximum U_{avg} inside that smaller optimum set. Such cases are possible with large capacities, which is why the submitted solver uses exact big-integer comparison instead of claiming that a fixed scale is always safe.

11.2 Weighted Exact Objective Mode

The implementation also provides an optional weighted exact mode. It minimizes

$$w_1R_1 + w_2R_2 + w_3Q - w_4U_{\text{avg}} - w_5U_{\min} + w_6O + w_7C,$$

where O is the number of assignments outside the submitted preference list and C is the number of students moved away from a supplied baseline assignment. The weights are nonnegative integers. The mode uses R_2, not the standard deviation itself, because R_2 is additive over student–laboratory edges. The reported standard deviation remains exact for the selected assignment.

The non-additive terms Q and $U_{\min}$ are handled outside the flow network as thresholds. The solver searches a two-dimensional space whose axes are max-rank thresholds q and minimum-fill lower-bound vectors induced by candidate ratios u. For a box

$$q \in [q_{\text{lo}}, q_{\text{hi}}], \qquad u \in [u_{\text{lo}}, u_{\text{hi}}],$$

it solves the relaxed corner $(q_{\text{hi}}, u_{\text{lo}})$ to obtain the additive lower bound $A(q_{\text{hi}}, u_{\text{lo}})$. The whole box is pruned only when

$$A(q_{\text{hi}}, u_{\text{lo}}) + w_3q_{\text{lo}} - w_5u_{\text{hi}}$$

cannot beat the incumbent weighted score. This branch-and-bound is exact: the rank constraint is only relaxed, the minimum-fill constraint is only relaxed, and the non-additive terms are bounded optimistically. Indeed, for any assignment x in the box,

$$q_{\text{lo}} \le Q(x) \le q_{\text{hi}}, \qquad u_{\text{lo}} \le U_{\min}(x) \le u_{\text{hi}}.$$

The relaxed corner $(q_{\text{hi}}, u_{\text{lo}})$ has a superset of the box's feasible assignments, so

$$A(q_{\text{hi}}, u_{\text{lo}}) \le A(x).$$

Because all weights are nonnegative,

$$w_3q_{\text{lo}} \le w_3Q(x), \qquad -w_5u_{\text{hi}} \le -w_5U_{\min}(x).$$

Adding these inequalities gives a lower bound on the full weighted score of every assignment in the box, so pruning cannot remove a better solution. When the common denominator L and the scaled costs fit safely in signed 64-bit arithmetic, the U_{avg} reward is folded into the ordinary edge costs by minimizing

$$(|\{j : c_j > 0\}|L)I - w_4 \sum_{j:c_j>0} n_j \frac{L}{c_j},$$

where I is the additive rank/outside cost. If that scaling is not safe, the solver falls back to the same least-common-denominator big-integer path comparator as the default exact tie-breaker. Thus U_{avg} is part of the primary weighted score and is never a floating-point approximation. When hard targets bound the rank axis above by $q_{\max}$, the scalar safety check for this weighted average-fill path only needs to consider rank costs up to $q_{\max}$, because no weighted corner solve can insert edges with larger ranks.

Average-fill hard targets are accepted only through exact bounded cases. If the existing laboratory lower bounds already imply

$$U_{\text{avg}} \geq \alpha,$$

then adding the target does not change the feasible region. If average fill is constant over complete assignments, for example when all positive capacities are equal, the target either rejects the whole instance as infeasible or again leaves the selected objective's feasible region unchanged. For supported single objectives, the remaining nontrivial cases use a bounded count-vector engine. Let L be the common denominator of the positive capacities and define

$$U_s(n) = \sum_{j:c_j>0} n_j \frac{L}{c_j}.$$

The hard target $U_{\text{avg}} \geq \alpha$ becomes one integer inequality on $U_s(n)$. The implementation enumerates laboratory count vectors satisfying all lower bounds, graph-capacity upper bounds, $\sum_j n_j = N$, and this integer resource inequality. For each retained vector, using n_j as the minimum count for laboratory j fixes the counts exactly because the lower bound sum is N. The solver then runs the appropriate exact min-cost-flow subproblem for the selected rank-first or fair objective and compares all count slices in that same objective order. If the common-denominator scaling or the exact count-vector bound is too large, the run is rejected rather than silently using a heuristic. Equivalent minimum-fill thresholds are compressed by their induced lower-bound-count vectors. Max-rank thresholds are compressed to the ranks that can actually change the student–laboratory edge set, together with the forced-placement sentinel rank $M + 1$. The best candidate is selected by exact integer comparison of the full weighted score. This is an exact optimizer for the stated weighted objective; it is not merely selecting among a small portfolio of precomputed objectives. The current implementation also seeds the incumbent with an exact rank/outside/change-only weighted solution evaluated under the full score, and processes search boxes in increasing lower-bound order with a binary heap. Both choices only improve pruning and do not change the set of candidate assignments or the proof of optimality. Relaxed-corner cache entries are allocated lazily, so unvisited (q, u) pairs cost only a pointer-table slot rather than a full big-integer score and copied solution.

12 Implementation Notes

The implementation is a single C11 file, `assignment.c`. The sequential solver core uses portable C data structures and arithmetic. Optional terminal confirmation and process-parallel portfolio execution use POSIX facilities when they are available; Windows/MinGW builds fall back to deterministic serial portfolio solving. Important implementation choices are:

- Dinic's algorithm for lower-bound feasibility.
- Successive shortest augmenting paths with potentials for min-cost flow.
- Layered initial potentials on fresh assignment graphs. Before any augmentation, positive residual capacity appears only on forward source–student/group, student/group–laboratory, and laboratory–sink edges, so topological relaxation gives the same initial distances as the general queue method. If this layer condition is violated, the implementation falls back to the general initialization.

- A custom 4-ary indexed heap whose keys are lexicographic cost tuples.
- Exact rational comparison for fill-rate candidates using an overflow-safe two-word product.
- Exact average fill-rate comparison in the final tie-break phase using fixed-limb unsigned big integers, least-common-denominator integer weights, and equal-capacity bucket compression.
- Exact pre-reservation of adjacency-list capacities before constructing each min-cost flow graph.
- A lower-bound priority queue for weighted-exact branch-and-bound boxes, so the most promising exact search region is examined first.
- Lazy weighted-exact relaxed-corner cache allocation; this keeps the exact cache but avoids eagerly allocating one large entry for every possible threshold pair.
- UTF-8 laboratory names are treated as byte strings; no character-level decoding is required for equality.
- Laboratory names are indexed with an open-addressed hash table, so parsing full preference lists does not repeatedly scan all laboratories with `strcmp`.
- Large input files are read by a buffered `fread()` line reader that preserves line-level validation semantics.
- Student duplicate detection uses an open-addressed hash table, and per-student duplicate preferences use generation stamps.
- Allowed-set groups are used in the threshold feasibility search, and active rank-row groups are used in the general min-cost-flow graph and expanded back to student-wise output after optimization.
- A non-fatal `try_solve_with_minimum_counts` path lets threshold searches use the min-cost-flow result itself as the feasibility test, avoiding a redundant Dinic graph when exact optimality does not require a separate precheck.
- Optional profiling counters record Dinic calls, min-cost-flow calls, exact-average comparisons, group builds, threshold candidates, graph sizes, and phase CPU timings.

12.1 Output Files

The main output file keeps the simple assignment format. The first line contains the number of students N. Each following line contains one student id and the assigned laboratory name:

```
student_count
student_id assigned_laboratory_name
student_id assigned_laboratory_name
...
```

Therefore the main output has exactly $N + 1$ lines. This is the only file written in the default submission mode.

If the optional `--reports` flag is specified, the program writes sidecar files. The first is named

`OUTPUT_FILE.metrics.txt`.

This file contains the numerical values of the five evaluation metrics: average assigned rank, rank standard deviation, maximum assigned rank, average fill rate, and minimum fill rate. It

also contains the rank sum, rank-square sum, per-laboratory assigned counts, and CPU-time measurements from the C standard `clock()` function. The extra file does not change the assignment-list format used by external evaluators.

The second sidecar file is named

`OUTPUT_FILE.by_lab.txt`.

It lists each laboratory, its assigned count, capacity, fill rate, fill percentage, and the student ids assigned there with their assigned ranks. A student assigned outside the submitted preference list is marked as `outside`.

The remaining sidecars are

`OUTPUT_FILE.by_student.tsv`

and

`OUTPUT_FILE.outside_preferences.tsv`.

They provide a tab-separated student-wise view and a focused list of students assigned outside their submitted preference list. These reports support faculty and student review while preserving the concrete student-wise output format used by external evaluators.

The report mode also writes

`OUTPUT_FILE.reasons.tsv`,

a compact per-student explanation label, including the first-choice demand, capacity, and assigned count used to justify congestion explanations.

When `--explain-student` or `--try-lock` is specified, the program also writes

`OUTPUT_FILE.explain.tsv`.

This file is an exact counterfactual explanation for one requested student–laboratory lock. The solver adds that lock, re-optimizes the remaining assignment under the selected objective, and reports feasibility, metric deltas, and the number of changed students. It is intentionally a single-objective tool: the program rejects counterfactual explanation combined with portfolio mode, because portfolio mode chooses among several objective orders and the re-solve would otherwise use a different decision rule from the selected assignment.

When `--profile` is specified, the program writes

`OUTPUT_FILE.profile.tsv`,

with internal solver counters and phase timing measurements. This file is not part of the assignment output; it is a reproducibility and tuning aid used to confirm whether an optimization removes actual work without changing the exact objective.

For adjustment runs with a previous assignment, the program writes

`OUTPUT_FILE.adjustment_delta.tsv`,

which compares baseline and adjusted metrics and lists moved students. Portfolio mode solves a light exact set of rank-oriented objective modes, writes

`OUTPUT_FILE.portfolio.tsv`,

and also writes candidate assignment files such as `OUTPUT_FILE.rubric.txt`, `OUTPUT_FILE.satisfaction.txt`, and `OUTPUT_FILE.fair.txt`. A deep portfolio option also includes heavier exact fill-focused, fill-convex, and weighted-exact candidates. The metrics and student reports include dissatisfaction summaries computed from the selected rank-cost model, so the

rank-oriented and satisfaction-oriented candidates can be compared side by side. The portfolio table includes the normalized components of the local recommendation score: average-rank component, rank-standard-deviation component, max-rank component, average-fill deficit, and minimum-fill deficit. This score is only a local selection proxy among generated candidates; an external relative-rank score is relative to all submitted programs. The table also records per-row `strengths` and `weaknesses` fields, which label the metrics where that generated candidate is relatively best or worst among the generated portfolio. When `--portfolio-summary-only` is specified, the program keeps the final assignment, the portfolio table, and ordinary reports but removes per-candidate assignment sidecars after they have been read.

The C program can also run portfolio candidates as independent child processes with `--jobs`. The repository includes a Python wrapper with the same purpose. This process-level parallelism does not alter the mathematical objectives or the selected candidate metrics; it only overlaps independent exact solver invocations on multi-core machines. The C implementation uses POSIX child processes when available and falls back to serial solving on Windows/MinGW. In parallel mode, reported solver CPU time is the sum of candidate CPU times; the observed wall-clock wait time can be smaller because the candidates run concurrently.

12.2 Operational Constraints and Adjustment

The optional constraints file supports hard constraints `lock student lab`, `forbid student lab`, `allow student lab...`, and `capacity lab value`. These are not heuristics. They restrict the feasible flow network, so the solver returns an exact optimum among assignments satisfying the stated constraints. With `--base-assignment` and `--change-penalty`, moving a student away from the supplied baseline adds an edge cost, which favors stable adjustments without changing the default output format.

12.3 Efficiency-Preserving Optimizations

The code contains several optimizations that do not change the mathematical objective.

First, laboratory-to-sink slots with the same cost are merged. Earlier, a laboratory with capacity c_j could be represented by c_j separate unit-capacity edges. This is unnecessary because all required slots have the same tuple cost and all optional slots have the same tuple cost. Therefore the implementation uses at most two edges:

$$\text{required edge capacity} = m_j, \qquad \text{optional edge capacity} = c_j - m_j,$$

where m_j is the current lower requirement for laboratory j. Sending k units through a capacity edge costs exactly the same as sending those k units through k identical unit edges, so the feasible assignments and lexicographic optimum are unchanged. The edge count is reduced substantially in dense cases.

Second, Dijkstra's priority queue is an indexed heap. Each vertex appears in the heap at most once, and a better distance performs a decrease-key update instead of inserting a duplicate record. The shortest paths are identical to ordinary Dijkstra with nonnegative reduced costs, but stale heap entries are avoided.

Third, if the total graph capacity is exactly N, then every feasible solution has fixed laboratory counts. The solver can skip the fill-rate search because $U_{\min}$ and U_{avg} are already determined by the hard constraints. Also, when the accepted $U_{\min}$ lower bounds sum to N, or when all positive capacities are equal, the average fill-rate tie-breaker cannot change the objective; the feasible solution already found during the $U_{\min}$ search is reused.

Fourth, the hot min-cost-flow edge cost stores only the three integer components that are used before the exact average-fill phase. Earlier versions kept a fourth scaled tie-break field in the same struct. After switching U_{avg} to exact least-common-denominator comparison, that field

became redundant. Removing it reduces edge memory traffic and speeds every min-cost-flow run without changing the mathematical objective.

Fifth, in the exact average-fill phase, laboratories with the same positive capacity are represented by one capacity term. This reduces big-integer weight construction, coefficient-vector copying, and exact heap-key comparison from the number of positive-capacity laboratories to the number of distinct positive capacities. The transformation is exactly the equal-denominator identity proved above.

Sixth, the threshold feasibility search groups students by allowed laboratory set for the tested threshold. The general min-cost-flow path then groups students with identical active rank rows for the fixed worst-rank bound. A group node has capacity equal to the number of students in the group, and its edges to laboratories use the representative student's rank costs. The grouped flow is expanded back to concrete student ids after optimization. This reduces the graph from N student nodes to $G \leq N$ group nodes without changing any objective value.

Seventh, repeated ungrouped min-cost-flow solves reuse an active-arc template when the problem pointer, constraint pointer, and rank threshold are unchanged. The template contains only the student–laboratory arc list and adjacency reservation counts. It does not contain edge costs, residual capacities, reverse edges, or laboratory lower-bound sink edges; all of those are rebuilt for each solve. Therefore the reuse cannot transfer residual flow state or an objective-specific cost from one threshold check to another. It only removes a duplicate scan of the active edge set.

Eighth, the rubric objective uses monotonicity in the minimum-fill threshold after the optimal average fill-rate has been fixed. Let $A(u)$ be the maximum average fill-rate achievable while preserving the already fixed rank-cost target and requiring $U_{\min} \geq u$. If $u_1 \leq u_2$, the feasible set for u_2 is a subset of the feasible set for u_1, so $A(u_1) \geq A(u_2)$. The implementation first computes $A(0)$, then finds the largest lower-bound vector that still achieves that same value using a short sequential prefix followed by exponential and binary search. Every accepted point is still solved by the exact flow optimizer, so this only removes redundant threshold probes.

Ninth, if all students have identical rank rows, the program uses a closed count-based solver. The check costs only $O(NM)$ comparisons, which is the same order as reading the rank table once. When the check succeeds, each student is interchangeable with every other student, so the assignment objective depends only on laboratory counts. The solver first fixes the rank-optimal count totals by rank group. Lower-rank groups are filled before higher-rank groups because each additional unit in a lower-rank group reduces R_1, and if R_1 ties, cannot increase R_2. Inside the one possible frontier group with equal rank, the program performs the same exact rational $U_{\min}$ search on counts, then assigns any remaining units to the smallest capacities first. This last step exactly maximizes $\sum_j n_j/c_j$, because each additional unit in laboratory j has marginal value $1/c_j$. For minimum-first objectives, average fill rate is used only after the minimum fill rate has been fixed. For rubric-style average-first objectives, the implementation instead preserves the best average-fill value while searching the minimum-fill threshold, as described above. Finally, the count vector is expanded back to the student-wise output format.

Tenth, the ordinary min-cost-flow Dijkstra queue uses a radix heap only when the current residual graph admits an exact unsigned scalar key. Before each augmentation, the solver scans positive-residual edges and computes their reduced costs. The radix path is enabled only if every reduced component (c_1, c_2, c_3) is componentwise nonnegative and if checked scales B_2 and B_1 fit such that

$$K(c) = c_1 B_1 + c_2 B_2 + c_3$$

preserves lexicographic order for every simple path in the current residual graph. A one-unit difference in c_2 is larger than the largest possible c_3 path difference, and a one-unit difference in c_1 is larger than the largest possible lower-component path difference. Thus Dijkstra's settled distances have monotone unsigned integer keys, satisfying the radix heap invariant. If any sign or overflow check fails, the previous binary heap is used without changing the path costs.

13 Complexity

Let E be the number of forward edges in the final min-cost flow network. In the worst case, student-laboratory edges are dense:

$$E = O(NM + N + M).$$

This bound uses the compressed laboratory-to-sink representation above. With $N \leq 1024$ and $M \leq 256$, this remains practical in C, and larger instances remain practical when the effective edge count is controlled by active grouping or repeated preferences. The min-cost flow sends at most N units of flow. Each augmentation performs an indexed-heap shortest path computation. The exact $U_{\min}$ phase adds a binary search over at most

$$\sum_j \min(c_j, N)$$

rational candidates. Under the target benchmark limits, this is at most $MN \leq 256 \cdot 1024 = 262144$; for larger OSS inputs, the same polynomial bound scales with the supplied M and N. The exact U_{avg} phase is run only once after the previous objectives are fixed, and is skipped when the average fill rate is already fixed by the counts or by uniform positive capacities. Let $P \leq M$ be the number of positive-capacity laboratories, let $K \leq P$ be the number of distinct positive capacities, and let B be the number of base-2^{31} limbs needed for the least-common-denominator weights W_d. Exact comparison of the average-fill component costs $O(KB)$ only when the first three integer cost components tie. This keeps the exact rational tie-breaker local to the final min-cost flow instead of slowing all previous feasibility and $U_{\min}$ searches.

If G_Q is the number of distinct active rank rows among students for the fixed worst-rank bound Q, the grouped min-cost-flow graph has G_Q group nodes and $O(G_Q M + G_Q + M)$ forward edges in the dense case. In the threshold search, if G_T is the number of distinct allowed laboratory sets for a tested threshold T, the feasibility graph has G_T group nodes instead of N student nodes. Since both G_Q and G_T are at most N, grouping cannot increase the graph size and can substantially reduce it when many students submit similar preference orders.

For the identical-rank fast path, the flow network is not built. The dominant work is sorting at most M laboratories and sorting at most $\sum_j \min(c_j, N)$ rational fill-rate candidates:

$$O(NM + M \log M + NM \log(NM)).$$

Under repeated-rank inputs this is much smaller than repeated shortest-path min-cost flow and is typically dominated by input and output time.

The implementation stores counts and graph indices in standard C `int` values, while large arrays use dynamically allocated storage. Therefore larger-than-benchmark inputs are not rejected merely for exceeding 256 laboratories or 1024 students, but the effective edge count still grows with the number of active groups times the number of laboratories. That product, together with available memory, is the main scaling limit.

14 Verification

The following checks were executed:

- `gcc -std=c11 -O2 -Wall -Wextra -Wshadow -Wconversion -pedantic -Werror`
- `python3 -m pytest -q`
- AddressSanitizer and UndefinedBehaviorSanitizer test run in which the pytest build hook is forced to compile the sanitizer-instrumented binary.

- Small-instance exhaustive search tests. These enumerate every assignment and verify that the solver's selected objective matches the true optimum. Weighted exact tests compare the exact scalar score itself, not an unrelated tie-break tuple.
- Maximum-scale stress input with $N = 1024$ and $M = 256$.

14.1 Dataset Suite

The repository contains the following concrete datasets under `datasets/`. Each dataset has a `_labs.txt` file and a `_prefs.txt` file.

Dataset	Purpose	Optimality evidence
`synthetic_sample`	Handout-format sample	General flow theorem
`forced_outside`	Forced outside preference	Exhaustive search
`zero_capacity`	Zero-capacity laboratory	Exhaustive search
`rank_variance`	Rank-sum and variance tie	Exhaustive search
`fill_rate`	Minimum fill-rate tie	Exhaustive search

For every small dataset, the test suite enumerates all possible assignments, filters exactly the feasible assignments, and compares the solver result against the exhaustive optimum for the selected objective mode. This includes the default rubric tuple

$$(R_1,\ R_2,\ Q,\ -U_{\text{avg}},\ -U_{\min}),$$

the fair tuple

$$(Q,\ R_1,\ R_2,\ -U_{\min},\ -U_{\text{avg}}),$$

satisfaction-cost tuples, and weighted-exact scalar scores. This proves optimality for those concrete datasets independently of the flow implementation. For larger datasets, exhaustive search is not computationally possible; optimality follows from the mode-specific theorems above, which apply to every valid input instance. The test suite also contains a huge-capacity case where a scaled integer average-fill tie-breaker would lose the exact ordering; the solver still matches the exhaustive optimum because the final comparison uses least-common-denominator big integers.

14.2 Comparison with Exhaustive Search

Exhaustive search is useful as a verifier for small inputs, but it is not a candidate for the submitted algorithm. It checks M^N assignments before capacity pruning. The following local timings compare a Python exhaustive checker with the C solver on the same small $M = 4$ family:

N	Assignments checked	Exhaustive search	C solver
6	4,096	0.0092s	0.0031s
8	65,536	0.2064s	0.0027s
10	1,048,576	4.0502s	0.0033s

At the benchmark size, direct enumeration would require

$$256^{1024} = 2^{8192} \approx 10^{2466}$$

candidate assignments before pruning. The flow formulation avoids this enumeration by using the assignment structure directly.

14.3 Performance

The maximum-scale stress runs used $N = 1024$ and $M = 256$. The table below is generated by `scripts/run_benchmarks.py`. Wall time is measured by the driver process, and CPU phase times are read from `--profile`. The values are machine-specific evidence rather than portable constants.

Case	Wall time	Solver CPU time	Notes
Uniform 256 labs / 1024 students, 8 preferences, rubric	2.022s	1.868s	reports/profile on
Popular-skewed 256 labs / 1024 students, 8 preferences, rubric	2.456s	2.400s	reports/profile on
Popular-skewed 256 labs / 1024 students, satisfaction	2.553s	2.456s	student-friendly rank costs
Popular-skewed 256 labs / 1024 students, fair	0.896s	0.850s	worst-rank first
Dense full preferences, 128 labs / 512 students, rubric	0.609s	0.598s	reports/profile on
Dense full preferences, 128 labs / 512 students, weighted evaluation	0.738s	0.726s	seed skipped on wide rank axis; 20 corner misses
Long-name full preferences, 256 labs / 1024 students, rubric	2.712s	2.458s	65.9MB input; read CPU 0.227s
Popular-skewed 256 labs / 1024 students, weighted rank-only	1.823s	1.805s	integer-only fast path
Popular-skewed 256 labs / 1024 students, weighted evaluation	8.288s	8.133s	priority-queue 2D branch-and-bound; 9 corner-cache hits
Popular-skewed 128 labs / 512 students, exact counterfactual	1.009s	0.507s	selected solver CPU; counterfactual CPU 0.492s

These cases are deliberately different: they include uniform preferences, popular-skewed preferences, dense full-preference input, a 65.9MB long-name input, weighted-exact threshold search, portfolio process parallelism, and counterfactual re-optimization. The optimizations are therefore not just a special-case shortcut. The slot compression and indexed heap affect every min-cost flow solve; the capacity and fill-rate shortcuts are used only when their mathematical preconditions make the skipped work redundant.

On a 128-laboratory / 512-student popular-skewed case, the built-in light portfolio took 1.895 seconds serially and 0.560 seconds with `--jobs 4`. With `--portfolio-summary-only`, the same light parallel portfolio took 0.565 seconds and removed per-candidate sidecars after reading them. The deep portfolio took 3.837 seconds serially and 1.448 seconds with `--jobs 4`. This process parallelism is an operational speedup only; each candidate is still produced by a normal exact `assign_labs` invocation. The reported solver CPU time for parallel portfolio mode is the sum of child-process solver CPU times, so it can be larger than wall time.

15 Conclusion

The solver does not enumerate assignments for the documented objective modes. Instead, for the stated assignment model and the selected structured objective, it uses total unimodularity and integrality of network flow, together with polynomial threshold enumeration and exact rational comparison, to obtain a global optimum exactly. This is not a claim of a universal polynomial-time optimizer for arbitrary assignment objectives. With only black-box access to an arbitrary objective, exact optimization can require exponential search: if some feasible assignment has not

been queried, two objectives can agree on every queried assignment while making that unqueried assignment uniquely optimal in one case and not optimal in the other.

With a succinct unrestricted objective, the model can encode SAT even while respecting the positive-capacity minimum-occupancy rule. For each Boolean variable x_i, create two laboratories T_i and F_i, each with capacity 2. Create one variable student X_i and two dummy students $D_{i,T}$ and $D_{i,F}$. Define the objective F_φ to score zero exactly when $D_{i,T}$ is assigned to T_i, $D_{i,F}$ is assigned to F_i, X_i is assigned to one of T_i, F_i, and the truth assignment induced by $X_i \mapsto T_i$ as true and $X_i \mapsto F_i$ as false satisfies the Boolean formula φ. All other assignments score one. The dummy students satisfy the one-student lower bound in both truth-value laboratories, and capacity 2 leaves room for the variable student. Hence φ is satisfiable if and only if $\min_a F_\varphi(a) = 0$. A polynomial-time exact optimizer for every such succinct objective would therefore decide SAT in polynomial time, implying $P = NP$. The implemented solver deliberately avoids this claim by supporting only structured objective families with flow, threshold, rational-comparison, or separable-convex structure.

The default mode is an exact Pareto-lexicographic flow solver for the objective order:

$$R_1 \to R_2 \to Q \to U_{\text{avg}} \to U_{\min}.$$

Additional objective modes, hard constraints, baseline-change penalties, and portfolio reports reuse the same exact flow core while preserving the required student-wise output format.